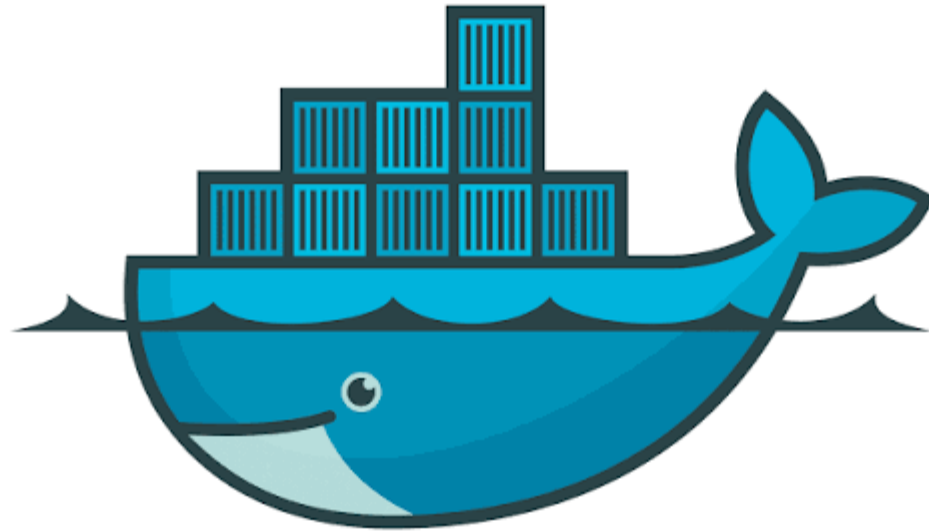
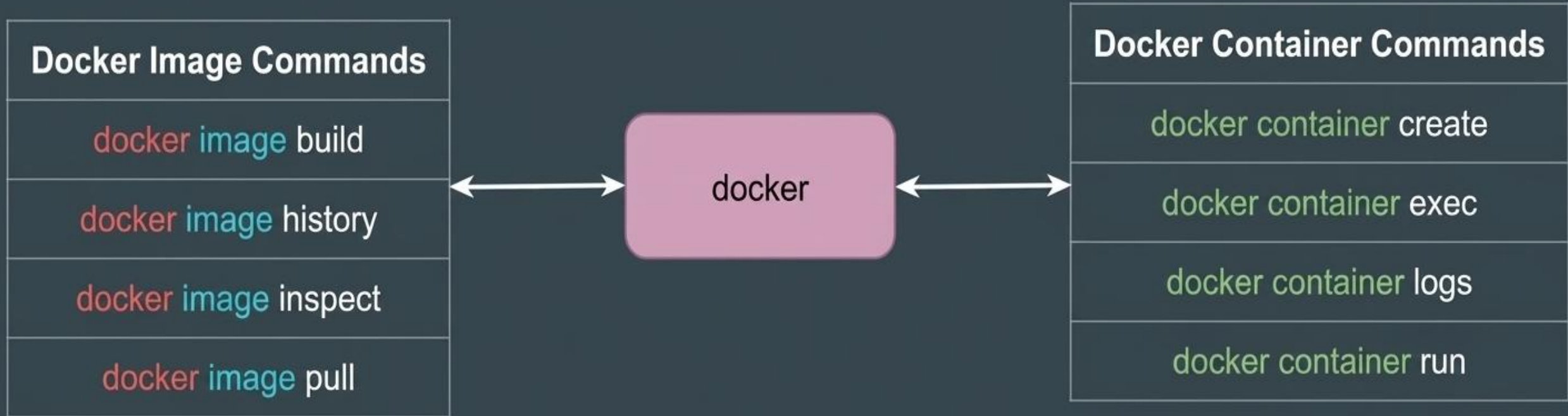


DevOps

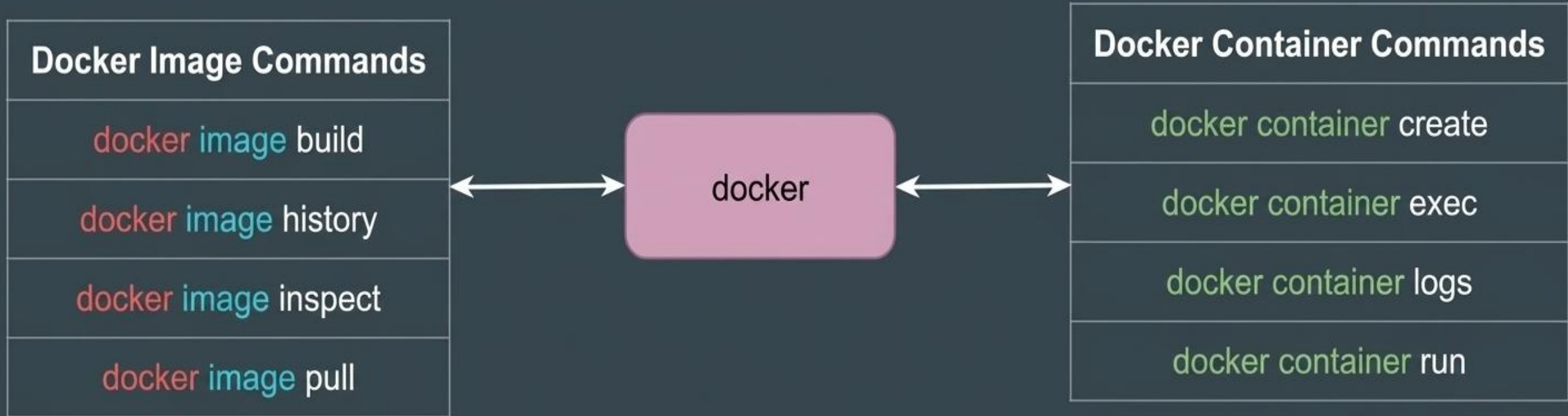


Docker

docker is the base command for Docker CLI



docker is the base command for Docker CLI



Docker Image from Dockerfile – Process & Rules

Write instructions → Build Image → Run Container

1 Write a Dockerfile



Dockerfile

```
FROM nginx:latest
WORKDIR /usr/share/nginx/html
COPY index.html .
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

A Dockerfile is a text file that contains a series of instructions to build a Docker image.

2 Build the Image

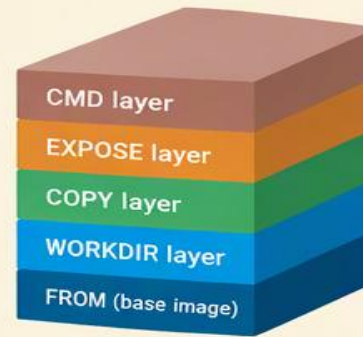
```
$ docker build -t my-nginx .
```



Docker Build

Docker reads the Dockerfile, executes each instruction in order and creates an image with multiple layers.

3 Docker Image Created



Docker Image

Each instruction creates a layer. The final image is a combination of all layers.

4 Run a Container

```
$ docker run -d -p 80:80 my-nginx
```



Running Container

A container is a running instance of the image.

Access your application:

<http://localhost:80>

Common Dockerfile Instructions

Instruction	Purpose
FROM	Specifies the base image (must be the first instruction)
RUN	Executes commands during image build
COPY	Copies files from host to image
ADD	Similar to COPY (supports URLs and auto-extracts tar)
WORKDIR	Sets the working directory
ENV	Sets environment variables
ARG	Defines build-time variables
EXPOSE	Documents the port the container listens on
CMD	Provides the default command to run the container
ENTRYPOINT	Defines the main executable to run
USER	Sets the user for subsequent instructions
VOLUME	Creates a mount point for external storage
LABEL	Adds metadata to the image
HEALTHCHECK	Defines a health check for the container

Rules for Writing a Dockerfile

- 1 The Dockerfile must begin with a FROM instruction.
- 2 Instructions are not case-sensitive, but convention is to use UPPERCASE (e.g., FROM, RUN).
- 3 Docker executes instructions in the order they appear.
- 4 Each instruction (that changes the filesystem) creates a new layer.
- 5 Use a .dockerignore file to exclude unnecessary files from the build context.
- 6 Keep each instruction as simple as possible and combine RUN commands to reduce layers.
- 7 Use specific image tags (e.g., python:3.11) instead of latest for reproducible builds.
- 8 Place less frequently changed instructions (e.g., installing packages) before frequently changed ones (e.g., copying source code) to leverage caching.
- 9 Use meaningful names, labels, and comments for maintainability.
- 10 Follow security best practices (avoid running as root, use minimal base images, etc.).

Important Notes

- FROM is mandatory and must be the first instruction.
- CMD and ENTRYPOINT define what runs when the container starts (use only one CMD, and one ENTRYPOINT).
- EXPOSE only documents the port (it does NOT publish it). Use -p with docker run to publish ports.
- COPY is preferred over ADD unless you need ADD's extra features (URL download or tar extraction).
- ENV variables are available in the container at runtime.
- ARG variables are only available during build (use --build-arg).
- A .dockerignore file works like .gitignore and excludes files from the build context.
- Use multi-stage builds for smaller and more secure images (especially for compiled applications).
- Each layer is cached. Docker reuses cached layers if the instruction and its context haven't changed.
- Use minimal base images (e.g., alpine, slim) to keep image size small.

"Dockerfile is code. Your image is the result. Follow the rules, build efficiently, and containerize confidently!"



Commands to write Docker File:

FROM	→ Selects the base image and starts a build stage.
RUN	→ Executes commands while building the image.
COPY	→ Copies files/directories into the image.
ADD	→ Similar to COPY, with additional capabilities such as handling local tar archives and remote resources.
WORKDIR	→ Sets the working directory for following instructions.
ENV	→ Defines environment variables.
ARG	→ Defines build-time variables.
EXPOSE	→ Documents the port the application listens on. It does not itself publish the port to the host.
USER	→ Selects the user/group used by subsequent instructions and container execution.
VOLUME	→ Creates a volume mount point.
ENTRYPOINT	→ Defines the container's default executable.
CMD	→ Provides the default command or arguments when the container starts.
HEALTHCHECK	→ Defines how Docker checks the health of the running container

Dockerfile

—	FROM	→ Where do I start?
—	WORKDIR	→ Where do I work?
—	COPY/ADD	→ What files do I need?
—	RUN	→ What should I install/build?
—	ENV/ARG	→ What variables do I need?
—	USER	→ Who runs it?
—	EXPOSE	→ Which port does the app use?
—	CMD/	
	ENTRYPOINT	→ What runs when container starts?

Example Dockerfile :

```
FROM python:3.12-slim
LABEL maintainer="setu"
LABEL application="docker-demo"
ARG APP_VERSION=1.0
ENV APP_ENV=production
WORKDIR /app
COPY app.py /app/app.py
ADD sample.txt /app/sample.txt
RUN pip install flask
EXPOSE 5000
VOLUME ["/app/data"]
USER root
ENTRYPOINT ["python"]
CMD ["app.py"]
HEALTHCHECK --interval=30s --timeout=3s \ CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:5000') || exit 1
```

→ # 1. FROM - Select the base image

→ # 2. LABEL - Add information/metadata about the image

→ # 3. ARG - Build-time variable

→ # 4. ENV - Environment variable available in the container

→ # 5. WORKDIR - Set the working directory

→ # 6. COPY - Copy files from host into the image

→ # 7. ADD - Copy/add files into the image

→ # 8. RUN - Execute commands while building the image

→ # 9. EXPOSE - Document the port used by the application

→ # 10. VOLUME - Define a volume mount point

→ # 11. USER - Select the user used to run commands/container

→ # 13. ENTRYPOINT - Main executable

→ # 14. CMD - Default argument supplied to ENTRYPOINT

→ # 12. HEALTHCHECK - Check whether the application is healthy



Docker Volumes

Persist your data. Keep it outside the container.

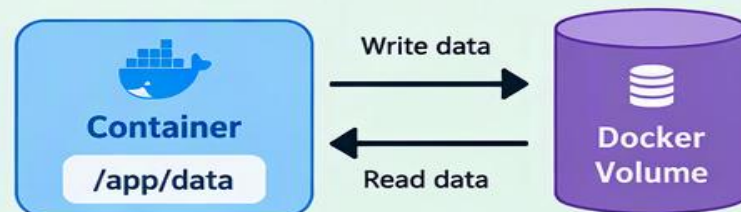


Containers are temporary.
Volumes make your data permanent.

Why Use a Docker Volume?

- ✓ Persist data beyond container lifecycle
- ✓ Keep data separate from the container
- ✓ Share data between multiple containers
- ✓ Easier backup, restore, and migration
- ✓ Useful for databases, logs, uploads, etc.

How It Works



Data is stored in a Docker volume on the host, not inside the container's writable layer.

Without vs With a Volume

Without a Volume



- Data is lost when the container is removed.
- Stored in the container's writable layer.

With a Volume



- Data persists even if the container is removed.
- Stored outside the container.

Common Operations

1 Create a Volume

```
# Create a named volume
docker volume create mydata
```

This creates a volume called mydata on your host.

2 Use a Volume in a Container

```
docker run -d \
  --name myapp \
  -v mydata:/app/data \
  nginx
```

This mounts the mydata volume to /app/data inside the container.

3 List Volumes

```
docker volume ls
```

Shows all available volumes.

4 Inspect a Volume

```
docker volume inspect mydata
```

Shows details about the volume, including its mount point on the host.

Types of Mounts

Named Volume (recommended)

- Managed by Docker
- Easier to use and portable
- Stored in Docker's volume directory

```
-v mydata:/app/data
```

Bind Mount

- Maps a host file or directory
- Good for development
- Uses an absolute host path

```
-v /host/path:/app/data
```

tmpfs Mount

- Stored in memory (not on disk)
- Data is lost when container stops
- Useful for sensitive or temporary data

```
--tmpfs /app/data
```

Example: Persistent Database

Run a MySQL container with a persistent volume.

```
docker volume create mysql-data
docker run -d \
  --name mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v mysql-data:/var/lib/mysql \
  mysql:8
```



Your database data will persist even if the container is removed.

Where Are Volumes Stored?

Docker stores volumes on the host machine, typically in:

```
/var/lib/docker/volumes/
```

(Exact location may vary by OS and Docker configuration.)

Useful Commands

```
docker volume ls      # List volumes
docker volume create <name> # Create a volume
docker volume inspect <name> # View details
docker volume rm <name> # Remove a volume
docker volume prune    # Remove unused volumes
```

Key Takeaways

- ✓ Use volumes to persist and share data.
- ✓ Containers are ephemeral, volumes are persistent.
- ✓ Named volumes are easiest and recommended.
- ✓ Great for databases, logs, user uploads, and more.
- ✓ Manage volumes with simple Docker commands.

Persistent
Data.
Happier
Containers!
😊



Docker Layers

Build once, reuse many times!

Docker images are made up of read-only layers, stacked on top of each other.

Each instruction in a Dockerfile creates a new layer.



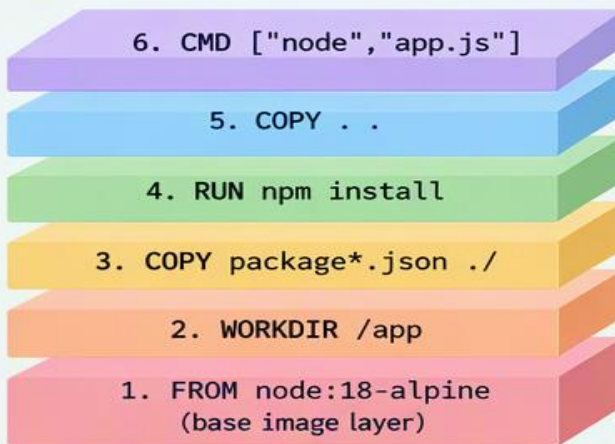
1 Dockerfile (example)

Each instruction creates a new layer.

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 CMD ["node", "app.js"]
```

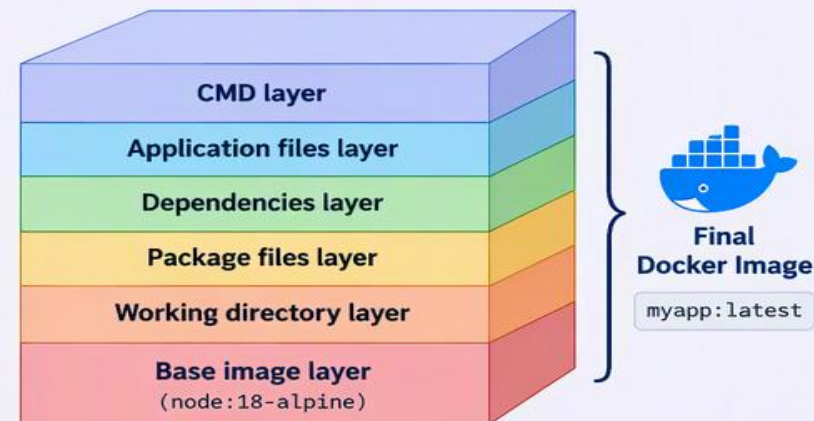
2 Layers Created

Each instruction adds a new read-only layer.



3 Final Image (Layered Structure)

All layers are stacked to create the final image.



Key Characteristics



Read-only

Layers are immutable. They cannot be changed after creation.



Stacked

Each new instruction adds a new layer on top of the previous one.



Cached

Docker reuses unchanged layers during builds (build cache).



Shared

Common layers (like base images) can be shared across multiple images.



Benefits of Layers



Faster Builds

Only changed layers need to be rebuilt. Unchanged layers are reused from cache.



Smaller Images

Shared layers reduce duplicate data across images.



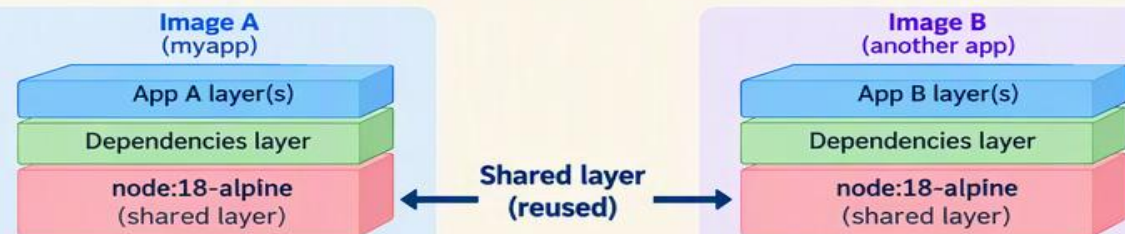
Efficient Distribution

Layers are reused when pulling/pushing images, saving bandwidth.



Layer Reuse Example

If two images use the same base image, they share the same base layer.



Things to Keep in Mind

1

Order matters

Place less frequently changed instructions (e.g., dependencies) earlier to get better cache reuse.

2

Minimize layers

Combine related commands (e.g., using &&) to reduce the number of layers.

3

Keep it clean

Remove temporary files in the same layer to avoid bloating the image.

4

Use .dockerignore

Exclude unnecessary files from the build context to avoid extra layers.

Smarter layers lead to smaller, faster, and more efficient images!





How to Reduce Docker Image Size

Smaller images = faster builds + quicker deployments + lower storage costs + reduced security risk

Lean Images
Happier Deployments

Before: A Larger Image

```
# Dockerfile (not optimized)
FROM node:18
WORKDIR /app
COPY : .
RUN npm install
CMD ["node", "app.js"]
```



Apply best
practices



Smaller, cleaner,
faster!

After: A Smaller, Optimized Image

```
# Dockerfile (optimized)
FROM node:18-alpine
WORKDIR /app
COPY package*.json :/
RUN npm ci --only=production
COPY : .
CMD ["node", "app.js"]
```



~ 120 MB
Image size

~ 90%
smaller!

Key Ways to Reduce Docker Image Size

1

Use a smaller base image

Choose minimal images (e.g., alpine) instead of full versions.

```
# Instead of
FROM node:18
# Use
FROM node:18-alpine
```

2

Install only production dependencies

Avoid installing dev dependencies in the final image.

```
# Node.js
RUN npm ci --only=production
# Python
RUN pip install --no-cache-dir -r requirements.txt
```

3

Use multi-stage builds

Build in one stage, copy only what you need to the final stage.

```
FROM node:18 AS builder
# build your app
FROM node:18-alpine
COPY --from=builder /app/dist /app
```

4

Minimize layers

Combine related commands to reduce the number of layers.

```
# Good: combine commands
RUN apt-get update && \
  apt-get install -y curl && \
  rm -rf /var/lib/apt/lists/*
```

5

Exclude unnecessary files

Use `.dockerignore` to prevent copying files like `node_modules`, `.git`, `logs`, etc.

```
# .dockerignore
node_modules
.git
*.log
.env
```

6

Clean up after installation

Remove package caches and temporary files in the same layer.

```
RUN apt-get update && \
  apt-get install -y curl && \
  rm -rf /var/lib/apt/lists/*
```

7

Use specific version tags

Avoid latest and use specific, minimal versions.

```
# Instead of
FROM python:latest
# Use
FROM python:3.11-alpine
```

8

Copy only what you need

Copy dependency files first (to leverage Docker cache), then copy the rest.

```
COPY package*.json ./
RUN npm ci --only=production
COPY . .
```

Extra Tools to Find and Reduce Size

- Check image size: `docker images`
- Analyze layers: `docker history <image-name>`
- Find large files inside: `docker run --rm -it <image> sh`
- Use tools: `dive <image-name>` (great for exploring layers)



Benefits

- ✓ Faster builds and deployments
- ✓ Lower storage and bandwidth costs
- ✓ Smaller attack surface (better security)
- ✓ Easier to distribute and scale



Remember

*"Only include what you need,
and nothing more."*



Smaller Images
Bigger Possibilities